

①

Hafiz M. Ahmad-Riaz

~~What~~

WHY OOPS ?

- C++ was designed with the main intentions of adding object-oriented features to C-language.
- As the size of the program increases, readability, maintainability and bug-free nature of programs decreases.
- This was the major problem with languages like C which relied on functions or procedures (hence the name procedural programming language).
- As a result, the possibility of not addressing the problem in an effective manner was high.
- Also, as data was almost neglected, data security was easily compromised.
- Using classes solves this problem by modelling program as a real world scenario.

Animal: → class

→ type	} variables
→ name	

→ Sound	} Methods
→ run	

* I use this template to make a **dog**.

We make tommy as a type of animal.

```
tommy = Animal
```

```
tommy.run();
```

```
tommy.sound();
```


Procedural Oriented Programming (POP):-

- Consists of writing a set of instructions for the computer to follow.
- Main focus on functions and not on flow of data.
- Functions can either use local or global data.
- Data moves openly from function to function.

```
main ( ) {
```

```
    f1 ( );
```

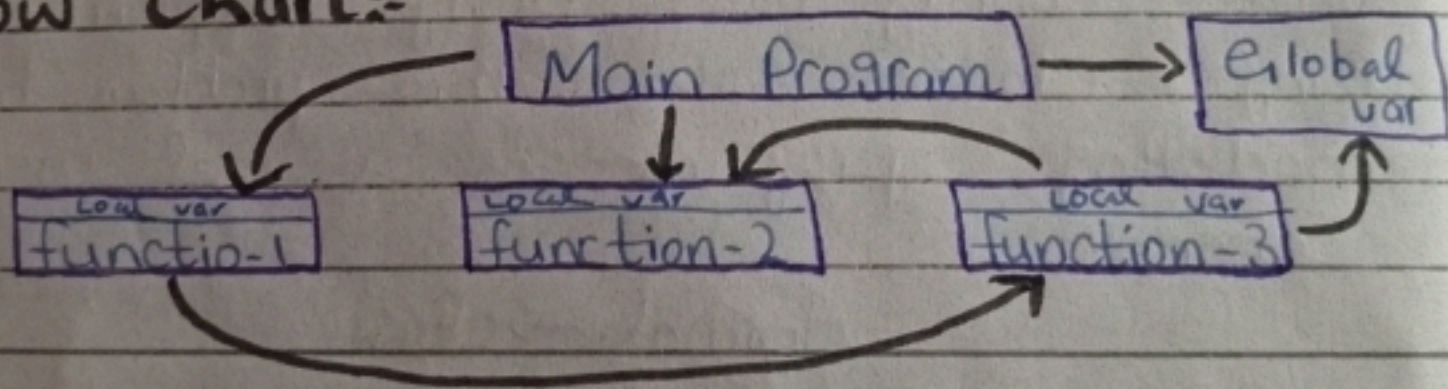
```
    f2 ( );
```

```
    f3 ( );
```

```
}
```

→ not in flow of data.

Flow chart:-



Object Oriented Programming (OOP):-

- Works on the concept of classes & objects.
- A class is a template to create objects.
- Treats data as a critical element.
- Decomposes the problem in objects & builds data & functions around the objects.

Class didn't take any place in memory.
When we make objects in class, then these objects take memory from system.

aming

- tions



- 8

a= Thing.

- 1

②

Class example:

```
#include <iostream>
using namespace std;
```

```
class Employee
```

{

```
private:
```

```
int a, b, c;
```

```
public:
```

```
int d, e;
```

```
void setData(int x, int y, int z);
```

```
void getData() {
```

```
cout << "value of a is" << a << endl;
```

```
cout << "value of b is" << b << endl;
```

```
cout << "value of c is" << c << endl;
```

```
cout << "value of d is" << d << endl;
```

```
cout << "value of e is" << e << endl;
```

}

};

```
void Employee :: setData(int x, int y, int z)
```

```
{
    a = x;
```

```
    b = y;
```

```
    c = z;
```

}

```
int main() {
```

```
    Employee Ahmad;
```

```
    Ahmad.setData(1, 5, 9);
```

```
    Ahmad.d = 100;
```

```
    Ahmad.e = 200;
```

```
    Ahmad.getData();
```

```
    return 0;
```

}


```
#include <iostream>
using namespace std;
```

```
class Shop
```

```
{ private
  Public:
```

```
    int itemId[100];
    int itemPrice[100];
    int counter;
```

```
  Public:
```

```
    void initCounter(void) { counter = 0; }
    void setPrice(void);
    void displayPrice(void);
```

```
}
```

```
void Shop::setPrice(void)
{
```

```
    cout << "Enter Id of your item" << counter + 1
    << endl;
```

```
    cin >> itemId[counter];
```

```
    cout << "Enter price of your item" << endl;
```

```
    cin >> itemPrice[counter];
```

```
    counter ++;
```

```
}
```

```
void Shop::displayPrice(void)
{
```

```
    for (int i = 0; i < counter; i++)
```

```
    {
```

```
        cout << "The Price of item with Id" <<
        itemId[i] << " is" << itemPrice[i] << endl;
```

```
    }
```

```
}
```



```

int main() {
    Shop* dukaan;
    dukaan.initCounter();
    dukaan.setPrice();
    //
    //
    dukaan.displayPrice();
    return 0;
}

```

③

- OOPS --> classes & objects.
- C++ --> Initially called --> "C" with class by Stroustrup.
- Class --> extension of structures (in C).
- Structures & Limitations
 - * Members are public.
 - * No methods.
- Classes --> Structures + More.
- classes --> Can have methods & Properties.
- classes --> can make few members as private & few as Public.
- Structures in C++ are typedefed.
- You can declare objects along with the class declaration like this:

```
/* Class Employee {
```

class definition

```
} ahmad, ali, saad;
```

ahmad.salary = 8 makes no sense if "salary" is Private.

"Nesting of Member Functions"

```
#include <iostream>
using namespace std;
#include <string>
```

```
class Binary
{
```

```
private:
```

```
    string s; void check_binary(void);
```

```
public:
```

```
    void read(void);
```

```
    void zero_ones_compliment(void);
```

```
    void display(void);
```

```
};
```

```
void Binary :: read(void)
```

```
{
```

```
    cout << "Enter a Binary number" << endl;
```

```
    cin >> s;
```

```
}
```

```
void Binary :: check_binary(void)
```

```
{
```

```
    for(int i=0; i<s.length(); i++)
```

```
{
```

```
        if(s.at(i) != '0' && s.at(i) != '1')
```

```
{
```

```
            cout << "Incorrect Binary Format" << endl;
```

```
            exit(0);
```

```
}
```

```
}
```

```
}
```



```
void Binary :: zero_ones_compliment(void)
```

```
{  
    check_binary();  
    for(int i=0; i<s.length(); i++)
```

```
{  
        if(s.at(i)=='0')
```

```
{  
            s.at(i)='1';
```

```
        }  
        else
```

```
{  
            s.at(i)='0';
```

```
        }  
    }
```

```
}
```

```
void Binary :: display(void)
```

```
{
```

```
    for(int i=0; i<s.length(); i++)
```

```
{
```

```
        cout<<s.at(i);
```

```
    }
```

```
    cout<<endl;
```

```
}
```

```
int main()
```

```
{  
    Binary b;
```

```
    b.read();
```

```
    b.display();
```

```
    b.zero_ones_compliment();
```

```
    b.display();
```

```
    return 0;
```

```
}
```


5

Static Data members & Methode Function:-

```
#include <iostream>
using namespace std;
```

```
class Person
```

```
{
    private:
```

```
        int id;
```

```
        static int count;
```

```
    public:
```

```
        void setData(void) {
```

```
            cout << "Enter Person Id" << endl;
```

```
            cin << id;
```

```
            count++;
```

```
        }
```

```
        void getData(void) {
```

```
            cout << "The Id of this person is << id <<
            "and this is person number"
            << count << endl;
```

```
        }
```

```
};
```

```
int Person :: count;
```

```
int main() {
```

```
    Person ahmad, ali, said; }
```

```
    ahmad.setData();
```

```
    ahmad.getData();
```

```
    ahmad.
```

```
    Person :: getCount();
```

```
    return 0;
```

```
static void getCount(void) {
```

```
    cout << "The value of
    count is " << count <<
    endl;
```

```
}
```


⑥ Array of Objects:-

```
#include <iostream>
using namespace std;
```

```
class Employee {
    int id;
    int salary;
```

```
public:
```

```
    void setId(void) {
```

```
        salary = 200;
```

```
        cout << "Enter your Id" << endl;
```

```
        cin >> id;
```

```
    }
```

```
    void getId(void) {
```

```
        cout << "The Id of this Employee is " <<
```

```
        id << endl;
```

```
    }
```

```
}; int main() {
```

```
    for int
```

```
        Employee no[4];
```

```
        for (int i = 0; i < 4; i++)
```

```
        {
```

```
            no[i].setId();
```

```
            no[i].getId();
```

```
        };
```

```
        return 0;
```

```
    }
```

Passing

```
#include <iostream>
using namespace std;
```

```
class G {
    private:
```

```
public:
```


Passing Objects as Function Arguments:-

```
#include <iostream>
using namespace std;
```

```
class Complex{
```

```
private:
```

```
int a;
```

```
int b;
```

```
public:
```

```
void setData(int a, int b){
```

```
    a = a;
```

```
    b = b;
```

```
}
```

```
void get setDataBySum(Complex x, Complex y)
```

```
{
```

```
    a = x.a + y.a;
```

```
    b = x.b + y.b;
```

```
}
```

```
void display_Sum(void){
```

```
    cout << "Your Complex Number is" << a <<
```

```
    " + " << b << " i" << endl;
```

```
}
```

```
} int main(){
```

```
    Complex r, t, u;
```

```
    r.setData(2, 3)
```

```
    r.setdisplay_Sum();
```

```
    t.setData(2, 3);
```

```
    t.display_Sum();
```

```
    u.setDataBySum(r, t)
```

```
    u.display_Sum();
```

```
    return 0;
```


⑦

Friend Functions in C++:-

```
#include <iostream>
using namespace std;
```

```
class Complex {
```

```
private:
```

```
    int a, b;
```

```
public:
```

```
    void setNumber(int x, int y) {
```

```
        a = x;
```

```
        b = y;
```

```
    }
```

```
    void displayNumbers() {
```

```
        cout << "The Number is " << a << " + " << b <<
            "i" << endl;
```

```
    }
```

```
};
```

```
Complex sum_Complex(Complex x1, Complex y1) {
```

```
    Complex z;
```

```
    z.setNumber(x1.a + y1.a, x1.b + y1.b);
```

```
    }
```

```
    return z;
```

```
int main() {
```

```
    Complex s, t;
```

```
    s.setNumber(1, 4)
```

```
    s.displayNumbers();
```

```
    t.setNumber(4, 1)
```

```
    t.displayNumbers();
```



```

result = SumComplex(c1, c2)
set result.displayNumbers();
return 0;
}

```

⑧ Friend classes Member functions

```

#include <iostream>
using namespace std;

```

```

class Complex;

```

```

class Calculator {

```

```

    public:

```

```

        int add(int a, int b) {
            return (a + b);
        }

```

```

        int to sumComplex(Complex, Complex);

```

```

    };

```

```

class Complex {

```

```

    private:

```

```

        int a, b;

```

```

        friend int Calculator::sumComplex(
            Complex, Complex);

```

```

    public:

```

```

        void add(int n, int y) {

```

```

            a = n;

```

```

            b = y;

```

```

        }

```


void display(void) {

cout << "The result is " << a << " + " << b << " i";
endl;

}

};

int Calculator::SumComplex(Complex x1, Complex x2)

{
return (x1.a + x2.a), (x1.b + x2.b);
}

int main() {

Complex r1, r2;

r1.add(2, 1);

r1.display();

r2.add(1, 2);

r2.display();

Calculator Cal;

int sum = Cal.SumComplex(r1, r2);

return

cout << "Result of two Complex number
is " << sum << endl;

return 0;

}

Q9

Simple program of Friend Function.

```
#include <iostream>
using namespace std;
class Saad;
class Ahmad {
private:
    int hobby1;
public:
    void setHobby(int first) {
        hobby1 = first;
    }
    friend void sum(Ahmad, Saad);
};
class Saad {
private:
    hobby2;
public:
    void setHobby(int second) {
        hobby2 = second;
    }
    friend void sum(Ahmad, Saad);
};
void sum(Ahmad a, Saad b) {
    cout << "Summing of Hobbies of Ahmad
    & Saad is " << a.hobby1 + b.hobby2 <<
    endl;
}
int main() {
```



```
Ahmad x;  
Saad y;  
x.setHobby('cricket');  
x;
```

```
Saad y;  
y.setHobby('examing');  
sum(x, y);  
return 0;
```

```
};
```

Complex program of Friend Functions

```
#include <iostream>  
using namespace std;
```

```
Class Swap_2;
```

```
Class Swap_1 {  
private:  
int val1;
```

```
public :
```

```
void Data(int a) {  
val1 = a;
```

```
}
```

```
void display(void) {
```

```
cout << val1 << endl;
```

```
}
```

```
};
```


class Swap2 {

private:

int val2;

public:

void Data(int a) {

~~cout << val2 << endl;~~ val2 = a;

}

void display(void) {

cout << val2 << endl;

}

};

void swap_value(swap_1 &x, swap_2 &y) {

int temp = x.val1;

x.val1 = y.val2;

y.val2 = temp;

}

int main() {

Swap_1 x1;

Swap_2 x2;

x1.Data(10);

x2.Data(20);

swap_value(x1, x2);

cout << "The value of x after swap is:";

x1.display();

cout << "The value of y after swap is:";

x2.display();

return 0;

⑩

Getting Started with Constructors

- Constructor is a special member function with the same name as of the class.
- It is used to initialize the objects of its class.
- It is automatically invoked whenever an object is created.

* Characteristics of Constructors *

- It should be declared in the public section of the class.
- They are automatically invoked whenever the object is created.
- They can have default arguments.
- They cannot return values & do not have return types.
- We cannot refer to their address.

Program:-

```
#include <iostream>
using namespace std;
```

```
Class Complex {
    int a, b;
```

```
public:
```

```
Complex(void);
```

```
void display();
```

```
{
    cout << "Enter no 13: " << a << " + " << b << "\n";
    endl;
```


Constructors:

function
class.

objects of

or an

Constructors *

section

the

not have

s.

+ "<< b << "i"

```
}  
Complex :: Complex(void) {  
    a = 0;  
    b = 0;  
}  
int main() {  
    Complex C1;  
    C1.display();  
    return 0;  
}
```

⑪ Point using Constructors:-

```
#include <iostream>  
using namespace std;
```

```
Class Point {  
    int x, y;  
    public:  
        Point(int a, int b) {  
            x = a; cout << "The point is (" << x << ", " << y  
            y = b; << ")" << endl;  
        }  
        void display(void) {  
            cout << "The point is (" << x << ", " << y  
            << ")" << endl;  
        }  
}  
int main() {  
    Point p(1, 0);
```



```

p.display();

// Point v(0,1);
v.display();

return 0;
};

```

Distance b/w two points:-

```

#include <iostream>
using namespace std;

```

```

class Point {

```

```

    int x, y; friend void distance(Point o1, Point o2);

```

```

public:

```

```

    Point(int a, int b) {

```

```

        x = a;

```

```

        y = b;
    }

```

```

    void display() {

```

```

        cout << "(" << x << ", " << y << ")" << endl;
    }

```

```

};

```

```

void distance(Point o1, Point o2) {

```

```

    float dist;

```

```

    dist = sqrt(pow(o2.x - o1.x, 2) + pow(o2.y - o1.y, 2));

```

```

    cout << dist << endl;
}

```



```
int main() {  
    Point p(0, 5);  
    p.display();
```

```
    Point q(0, 1);  
    q.display();
```

```
    distance(p, q);
```

```
    return 0;
```

```
}
```

⑫ Constructor Overloading:-

```
#include <iostream>
```

```
using namespace std;
```

```
class Complex {
```

```
    int x, y;
```

```
public:
```

```
    Complex() {
```

```
        x = 0;
```

```
        y = 0;
```

```
    }
```

```
    Complex(int a) {
```

```
        x = a;
```

```
        y = 0;
```

```
    }
```

```
    Complex(int a, int b) {
```

```
        x = a;
```

```
        y = b;
```

```
    };
```

```
    void display() { cout << x << " + " << y << "i" << endl;
```

```
};
```

```
}
```



```

int main() {
    Complex c1(1, 2);
    c1.display();
    Complex c2(1);
    c2.display();
    Complex c3;
    c3.display();

```

```

    return 0;
}

```

⑭ Bank Interests:-

```

#include <iostream>
using namespace std;

```

```

class BankDeposit
{

```

```

    int principal;
    int year;
    float interestRate;
    float returnValue;

```

```

public:

```

```

    BankDeposit(){};
    BankDeposit(int p, int y, int float x);
    BankDeposit(int p, int y, int x);
    void show_amount();
};

```



```

BankDeposit :: BankDeposit(int p, int y, float x){
    principal = p;
    year = y;
    interestRate = x;
    returnValue = principal;
    for(int i=0; i<y; i++)
    {
        returnValue *= (1+interestRate);
    }
}

```

```

BankDeposit :: BankDeposit(int p, int y, int x){
    principal = p;
    year = y;
    interestRate = float(x)/100;
    returnValue = principal;
    for(int i=0; i<y; i++)
    {
        returnValue *= (1+interestRate);
    }
}

```

```

void BankDeposit :: show(){
    cout << "Principal amount was " << principal
    << ". Return value after " << year " years
    is " << returnValue << endl;
}

```

```

int main(){
    BankDeposit bd1, bd2, bd3;
    int p, y, R;
    float x;
}

```



```

cout << "Enter principal amount, year and
        interest rate in decimal:" << endl;
cin >> p, y, r; bd1 = BankDeposit(p, y, r);
bd1.show();

```

```

cout << "Enter principal amount, year and
        interest rate in percentage:" << endl;
cin >> p, y, R; bd2 = BankDeposit(p, y, R);
bd2.show();

```

```

    return 0;
}

```

⑮ Copy Constructor:-

```

#include <iostream>
using namespace std;

```

```

class Copy {
    int num;
public:
    Copy() {
        num = 0; // Default constructor
    }
    Copy()
    Copy(int n) { // Parameterized
        n = num;
    }
    Copy
    Copy(Copy &a) { // Copy Constructor
        num = a.num;
    }
}

```


ar and
endl;

```
void display() {  
    cout << "number is: " << num << endl;
```

```
};
```

s and
endl;

```
int main() {  
    copy b, c, d(10);  
    b.display();  
    c.display();  
    d.display();
```

```
    Number x = d;  
    x.display();
```

```
    Number a(b);  
    a.display();
```

```
    return 0;  
}
```

structor.

sized "

structors

16

* Destructors :-

- Destructors never takes an argument. nor does it return any value.

```
#include <iostream>
using namespace std;
```

```
class number {
```

```
public:
```

```
number () {
```

```
// Calling Constructors
```

```
    cout << ++;
```

```
    cout << "This is the time when my  
    constructor is called for an  
    object number" << endl;
```

```
}
```

```
// Calling Destructors.
```

```
~number () {
```

```
    cout << "This is the time when  
    my Destructors is called  
    for an object number" <<  
    endl;
```

```
    cout << --;
```

```
}
```

```
};
```

```
int main () {
```

```
    cout << "We are inside our main function" <<  
    endl;
```

```
    cout << "We are creating an object n1" << endl;
```



```
numbox m1;  
{
```

```
cout << "Entering this block" << endl;  
cout << "Creating two more objects" << endl;  
numbox n2, n3;
```

```
cout << "Exiting this block" << endl;  
}
```

```
cout << "Back to main function" << endl;
```

```
return 0;  
}
```

(17)

Inheritance in C++ :-

- Reuseability is a very important feature of OOPS.
- In C++, we can reuse a class and add additional features to it.
- Reusing classes saves time and money.
- Reusing already tested and debugged class will save a lot of effort of developing & debugging the same thing again.

What is inheritance :-

- The concept of reusability in C++ is supported by using inheritance.
- We can reuse the properties of an existing class by inheriting from it.
 e.g. Employee.
 ↑
 Base class.
- The existing class is called the **Base** class.
- The new class which is inherited is called

eg: Programmer.

- as the **Derived** class.
- Reusing classes saves time & money.
 - There are diff types of inheritance in C++.

Types:-

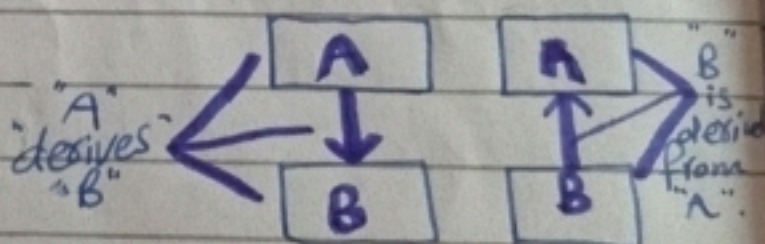
- There are **diff** types of inheritance in C++:-
- i) Multiple inheritance.
 - ii) Hierarchical inheritance.
 - iii) Multi-level inheritance.
 - iv) Hybrid inheritance.
 - v) Single inheritance.

(i)

Single Inheritance:-

One derived class which is inherit from one base class is called single inheritance.

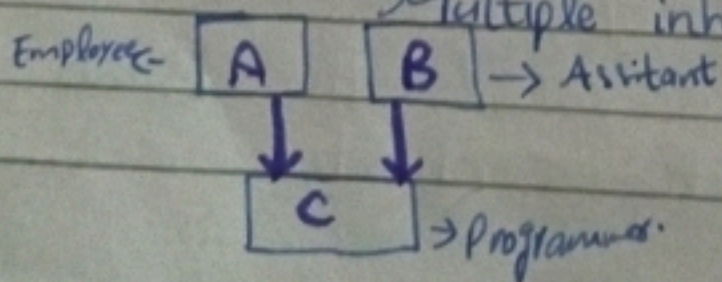
Employee → Programmer



(ii)

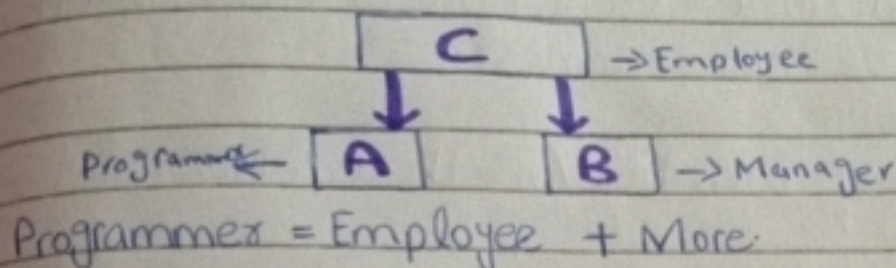
Multiple Inheritance:-

A derived class which is inherit from more than one base class is called "Multiple inheritance".



(iii)
Hierarchical Inheritance :-

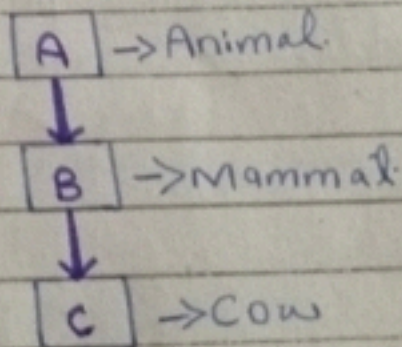
Different derived classes which are inherit from a single Base class is called "Hierarchical inheritance".



(iv)
Multilevel Inheritance :-

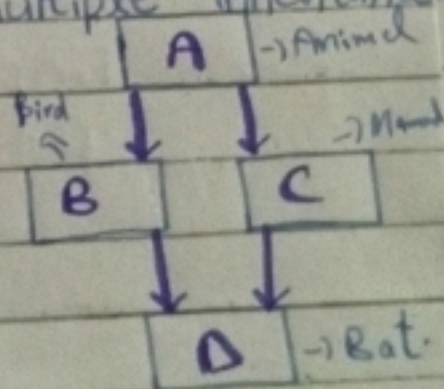
Deriving a class from already derived class.

Mammal = Animal + More.
Cow = Mammal + More.



(v)
Hybrid Inheritance :-

- It is a combination of multiple inheritance and mult-level inheritance.
- A class is derived from two classes as in multiple inheritance.
- However, one of the parent classes is not a base class.



(18)

Inheritance syntax & visibility mode

Syntax.

```
class Program  
class { {derived-class-name} } : { {visibility-mode}  
    { {base-class-name}  
    {  
        class members/methods/etc.....  
    }
```

Key Points:-

- **Default visibility mode:** ^{It} is private.
- **Public visibility mode:** Public members of the base class becomes public members of derived class.
- **Private visibility mode:** Public members of the base class becomes private members of derived class.
- **Private members:**

These members are never inherited.

"Program"

```
#include <iostream>  
using namespace std;
```

```
class Employee {  
    public:  
        int id;  
        float salary;
```



```
Employee(int inpId){  
    int id = inpId;  
    salary = 11.0;  
};
```

// creating a programmer class derived from Employee Base class.

```
class Programmer : public Employee {  
    Programmer(int inpId){  
        id = inpId;  
    }  
    int languageCode = 10;
```

```
    void print(void){  
        cout << id << endl;  
    }  
};
```

```
int main(){  
    Employee ahmad(1), saad(2);  
    cout << ahmad.salary << endl;  
    cout << ahmadsaad.salary << endl;
```

```
    Programmer detail(8);  
    cout << detail.languageCode << endl;  
    detail.print();
```

```
    return 0;  
};
```


(19)

Single Inheritance :-

```
#include <iostream>
using namespace std;
```

```
class Base {
    int num1;
public:
    int num2;
    void set_num();
    void get_num1();
    void get_num2();
};
```

```
void Base :: set_num(void) {
    Data num1 = 10;
    num2 = 20;
}
```

```
int Base :: get_num1() {
    return num1;
}
```

```
int Base :: get_num2() {
    return num2;
}
```

```
class Derived : public Base {
    int Data num3;
public:
    int process();
    void display();
};
```



```

int Derived = process() {
    int num3 = int num3 * get_num1();
}

void display() {
    cout << "The value of num1 is: " << get_num1() << endl;
    cout << "The value of num2 is: " << num2 << endl;
    cout << "value after calculation is: " << num3 << endl;
}

int main() {
    Derived der;
    der.set_num();
    der.get_process();
    der.display();

    return 0;
}

```

② Protected Access Modifier in C++:-

A private variable who is inherited is called protected.

Important Table:-

	Public Derivation	Private	Protected
1) Private members:	Not inherited	Not inherited	Not inherited
2) Protected members:	Protected	Private	Protected
3) Public members:	Public	Private	Protected

(1)

```
#include <iostream>
using namespace std;

class Base {
    protected:
        int password;
    private:
        int email;
};

class Derived : protected Base {
};
```

```
int main() {
    Base b;
    Derived d;
    // cout << d.a; // will not work bcz a is
    // protected in base as well as derived.
```

(2)

Note:-

If we are inheriting B from A & C from B
[A → B → C]:

- 1) A is the base class for B & B is the base class for C.
- 2) A → B → is called inheritance Path.

multilevel inheritance:-

```
#include <iostream>
using namespace std;
```

```
class Student {
    protected:
        int roll_no;
    public:
        void set_roll_no(int);
        void get_roll_no(void);
};
```

```
void Student::set_roll_no(int x) {
    roll_no = x;
}
```

```
void Student::get_roll_no(void) {
    cout << "Your roll no is: " << roll_no << endl;
}
```

```
class Exam : public Student {
    protected:
        float maths;
        float physics;
    public:
        void set_marks(float, float);
        void get_marks(void);
};
```

```
void Exam::set_marks(float m, float p) {
    maths = m;
    physics = p;
}
```



```

void Exam::get_marks() {
    cout<<"Your marks in maths are: "<<marks<<endl;
    cout<<"Your marks in physics are:"<<physics<<endl;
}

```

```

class Result : public Exam {
protected:
    float percentage;
public:
    void display_result(void) {
        get_roll_no();
        get_marks();
        cout<<"Your marks in percentage are"
        << percentage (marks + physics) / 2 <<
        "%" << endl;
    }
};

```

```

int main() {
    Result res;
    res.set_roll_no(520);
    res.set_marks(90, 66);
    res.display_result();

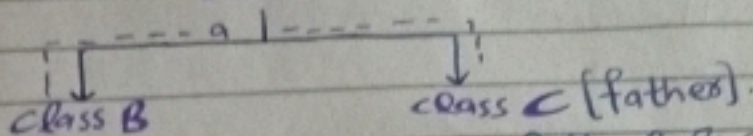
    return 0;
}

```


(24)

Virtual Base class:-

Class A [Grandfather]



Both B & C should try to inherit "a" at one only.

```

class B: virtual public A {
};

class C: virtual public A {
};
  
```

Program:-

```

#include <iostream> #include <string>
using namespace std;
  
```

```

class Student {
protected:
    int roll_no;
public:
    void set_roll_no(int x) {
        roll_no = x;
    }
    void get_roll_no(void) {
        cout << "Your roll no is:" << roll_no << endl;
    }
};
  
```

```

class Exam: virtual public Student {
protected:
    int maths, physics;
  
```



```

public:
    void set_marks(int m, int p) {
        maths = m;
        physics = p;
    }

```

```

    void get_marks(void) {
        cout << "Your marks in Maths are: " << maths
        << endl << "Your marks in physics
        are: " << physics << endl;
    }

```

```

};

```

```

class Hobby : public virtual Student {

```

```

protected:
    intstring hobby;

```

```

public:
    void set_hobby(intstring h) {
        hobby = h;
    }

```

```

    void get_hobby(void) {
        cout << "Your favorite hobby is: " << hobby << endl;
    }

```

```

};

```

```

class Total : public Exam, public Hobby {

```

```

protected:
    int result;

```

```

public:
    void set_result(int x) {
        result = x;
    }

```



```
void get_result(void) {  
    get_roll_no();  
    get_marks();  
    get_hobby();  
}
```

```
cout << "Total result is: " << endl  
    << "Student Roll No: " << roll_no << endl  
    << "Student marks: " << math << " " << physics  
    << endl << "Student Hobby is: " << hobby  
    << endl;
```

```
}
```

```
int main() {  
    Total t;  
    t.set_roll_no(520);  
    t.set_marks(90, 66);  
    t.set_hobby("cricket");  
    t.get_result();  
}
```

```
    return 0;  
}
```


(26)

* Constructors in Derived class :-

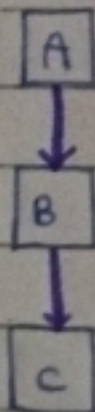
- We can use constructors in derived classes in C++.
- If base class constructor does not have any arguments, there is no need of any constructors in derived class.
- But if there are one or more arguments in the base class constructor, derived class need to pass arguments to the ^{base} class constructor.
- If both Base and Derived classes have constructors, base class constructor is executed first.

* Constructors in Multiple inheritance.

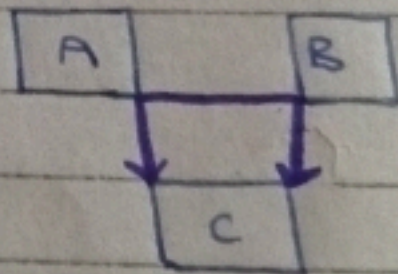
- In Multiple inheritance, base classes are constructed in the order in which they appear in the class declaration.

* Constructors in Multi-level inheritance :-

- In Multilevel inheritance, the constructors are executed in the order of inheritance.



Multilevel inher.



Multiple inher.

"Special Syntax"

- C++ supports a special syntax for passing arguments to multiple base classes.
- The constructor of the derived class receives all the arguments at once and then will pass the calls to the respective base classes.
- The body is called after all the constructors are finished executing.

Syntax:-

```
Derived-Constructor (arg1, arg2, arg3) : Base1-Constructor  
    (arg1, arg2), Base2-Constructor (arg3, arg4)  
{  
    -----  
}
```

```
Base1-Constructor (arg1, arg2).
```

e.g:-

```
C obj(7, 8, 9, 10, 11, 12)
```

```
C(a1, b1, b2, c1, c2) : B(b1, b2), A(a1, a2)  
{  
    -----  
}
```

```
B(b1, b2)
```

Special case of Virtual Base class:-

- The constructors for virtual base classes are invoked before a non-virtual base class.
- If there are multiple virtual base classes, they are invoked in the order declared.
- Any non-virtual base class are then constructed before the derived class constructor is executed.

(28)

Initialization list in Constructors

- Initialization list is nothing special, But it just pass a value to a member of class by Constructors.

Syntax Constructor (arg-list) : initialization-section {
assignment+others
}

e.g.

```
#include <iostream>
using namespace std;
```

Class Number {

int a, b; // private by default.

public:

Number(int a, int b) : a(a), b(b) {

cout << "Enter the value of a << a << endl;

cin >> a;

cout << "Enter the value of b << endl;

cin >> b;

cout << "The sum is: " << a+b << endl;

} -> void display(void) {

};

int main() {

Number n(0,0);

n.display();

} return 0;

(29)

Pointer

value is the same

e.g.

#include

using

int m

②9

Pointers :-

A pointer is a variable whose value is the address of another variable of the same type.

e.g.:-

```
#include <iostream>
using namespace std;
```

```
int main() {
    int a = 4;
    int *ptr = &a;
    cout << "The value of
    a is: " << *ptr << endl;
    cout << "The address of a
    is: " << &a << endl;
    return 0;
}
```

use of new keyword :-

```
#include <iostream>
using namespace std;
```

```
int main() {
    float *p = new float(40.9);
    cout << "The value at address
    p is: " << *(p) << endl;
    return 0;
}
```

Use of delete keyword in pointers with array.

```
#include <iostream>
using namespace std;
```

```
int main() {
    int *arr = new int[2];
    arr[0] = 30;
    arr[1] = 300;
    delete arr;
    cout << "The value at
    arr[0] is: " << arr[0];
}
```

```
return 0;
}
```


Pointers to objects & Arrow func.

```
#include <iostream>
using namespace std;
```

```
class Number {
    int n1, n2;
```

```
public:
```

```
void setData(int a, int b) { n1=a; n2=b; }
```

```
n1=a;
n2=b;
```

```
void getData(void) {
```

```
    cout << "The value of n1 is: " << n1 << endl;
```

```
    cout << "The value of n2 is: " << n2 << endl;
```

```
}
```

```
}
```

```
int main() {
```

```
    Number *ptr = &c;
```

```
    (*ptr).setData(1, 6);
```

```
    (*ptr).getData();
```

```
// OR we use:
```

```
    Number *ptr = new Number;
```

```
    (*ptr).setData(1, 6);
```

```
    (*ptr).getData();
```

```
// We use "ptr->" instead of "(*ptr)." which  
// is called Arrow function.
```

```
    Number *ptr = new Number;
```

```
    ptr->setData(1, 6);
```

```
    ptr->getData();
```

```
    return 0;
```

```
}
```